

Lab 04: Report Design UX

Lab 04: Advanced Report Design and User Experience

Lab summary

These labs guide learners through building an interactive report experience using the semantic model and measures created in earlier modules.

Novice-friendly how-to guide

Create a drillthrough page

1. Add a new report page.
2. Rename it `Customer Detail`.
3. In the Visualizations pane, find the **Drill-through** field well.
4. Drag `DimCustomer[CustomerName]` into the drill-through field well.
5. Add customer KPI cards and a transaction table.
6. Add a Back button from **Insert > Buttons > Navigator > Back**.
7. Test from a summary page by right-clicking a customer value and choosing the drillthrough page.

Create a tooltip page

1. Add a new report page.
2. Open page formatting and set the page type to **Tooltip**.
3. Add compact KPI or trend visuals.
4. Select the visual that should use the tooltip.
5. In the visual formatting settings, assign the tooltip page.

Create a field parameter

1. Select **Modeling > New parameter > Fields**.
2. Name the parameter `Metric Parameter`.
3. Select measures such as `[Sales Amount]`, `[Gross Margin]`, `[Gross Margin %]`, and `[Quantity]`.
4. Keep **Add slicer to this page** selected and choose **Create**.
5. Add the generated parameter field to a visual's values well.
6. Use the slicer to switch which measure the visual displays.

Azure Government readiness

The required labs are **Gov-ready** because they use core Power BI Desktop report features. Personalized visuals are **Verify for Gov**. All visuals are **Verify for Gov / Commercial-focused** and should not be required in Gov delivery unless validated.

Power BI project format

Build report artifacts as PBIP projects. PBIP is the source-controlled format for this workshop. PBIX files can be generated from PBIP later if a packaged file is needed.

Prerequisites

- Lab 01 semantic model.
- Lab 02 DAX measures.
- Power BI Desktop.

Concept note: advanced UX principles

Treat each report page as a decision interface. Put the audience's question first, reduce cognitive load with clear page structure, and use interactivity only when it helps the user decide what to do next. Avoid adding slicers, buttons, or visuals that compete with the primary task.

Concept note: audience-driven pages and filters

Use different page purposes for different audiences: executives need high-level status and exceptions, analysts need exploration paths, and operational users need monitoring and detail. Place slicers consistently, prefer clear page-level filters for shared context, and avoid hidden filter combinations that make results hard to explain.

Exercise 1: Drillthrough

Objective: Add a detail path from summary analysis to entity detail.

Tasks

1. Create a page named `Customer Detail`.
2. Add `CustomerName` as a drillthrough field.
3. Add customer-level KPI cards and a transaction table.
4. Add a Back button.
5. Test drillthrough from a customer visual on the summary page.

Expected result

Users can right-click a customer and navigate to a filtered detail page.

Exercise 2: Report page tooltips

Objective: Add contextual detail without cluttering the main report page.

Tasks

1. Create a page named `Sales Tooltip`.
2. Set page size to Tooltip.
3. Add compact KPIs and a small trend visual.
4. Enable the page as a tooltip.
5. Assign the tooltip page to a main report visual.

Expected result

Hovering over the assigned visual displays compact contextual information.

Exercise 3: Bookmarks and buttons

Objective: Create guided interactions.

Tasks

1. Create an information panel or slicer panel.
2. Add Show and Hide bookmarks.
3. Add buttons that trigger the bookmarks.
4. Review bookmark options for Data, Display, and Current page.
5. Add a Reset button if appropriate.

Expected result

Users can reveal and hide guided content without leaving the page.

Exercise 4: Dynamic navigation

Objective: Improve report navigation.

Tasks

1. Add page navigation buttons.
2. Add a report page navigator where appropriate.
3. Use clear labels and consistent placement.
4. Test navigation from every page.

Expected result

Users can move through the report without relying only on page tabs.

Exercise 5: Field parameters for guided exploration

Objective: Let report users switch between business metrics and dimensions without duplicating report pages.

Tasks

1. Confirm measures for Sales Amount, Gross Margin, Gross Margin %, and Quantity exist.
2. Create a measure field parameter named `Metric Parameter`.
3. Add the field parameter to a slicer.
4. Use the selected parameter in a visual.
5. Optional: create a dimension field parameter named `Dimension Parameter` for Product Category, Territory, and Customer Segment.

How-to guide

1. In Power BI Desktop, select **Modeling > New parameter > Fields**.
2. Name the parameter `Metric Parameter`.
3. Select `[Sales Amount]`, `[Gross Margin]`, `[Gross Margin %]`, and `[Quantity]`.
4. Keep **Add slicer to this page** selected and select **Create**.
5. Rename the generated slicer title to `Select Metric`.
6. Add a bar chart, line chart, or matrix to the page.
7. Drag the generated `Metric Parameter` field into the visual values well.
8. Use the slicer to switch which measure appears in the visual.
9. To create a dimension switcher, repeat **Modeling > New parameter > Fields**, name it `Dimension Parameter`, and select fields such as `DimProductCategory[ProductCategory]`, `DimTerritory[TerritoryRegion]`, and `DimSegment[Segment]`.
10. Add the generated `Dimension Parameter` field to the axis or rows area of a visual.

Expected result

Learners can provide controlled self-service flexibility without creating duplicate report pages.

Deeper Understanding Challenge: Build a disconnected metric selector

Field parameters are the native Power BI way to let users switch fields or measures in a visual. Another common pattern is a disconnected selector table plus a DAX `SWITCH` measure. Build this second pattern so you can compare it to the native field parameter.

Goal

Create a metric selector using Power Query and DAX, then compare it to the field parameter created in the main lab.

Step 1: Create the disconnected selector table in Power Query

1. Open **Transform data**.
2. Create a **Blank Query**.
3. Rename it `Metric Selector`.
4. Open **Advanced Editor**.
5. Replace the query with:

```

let
    Source =
        #table(
            type table [Metric = text, SortOrder = Int64.Type],
            {
                {"Sales Amount", 0},
                {"Gross Margin", 1},
                {"Gross Margin %", 2},
                {"Quantity", 3}
            }
        )
in
    Source

```

6. Select **Close & Apply**.

7. In Model view, confirm **Metric Selector** has no relationships.

8. Sort **Metric Selector[Metric]** by **Metric Selector[SortOrder]**.

Step 2: Create the DAX switching measure

```

Selected Metric Value =
VAR SelectedMetric =
    SELECTEDVALUE ( 'Metric Selector'[Metric], "Sales Amount" )
RETURN
    SWITCH (
        SelectedMetric,
        "Sales Amount", [Sales Amount],
        "Gross Margin", [Gross Margin],
        "Gross Margin %", [Gross Margin %],
        "Quantity", [Quantity],
        [Sales Amount]
    )

```

Step 3: Use it in a visual

1. Add **Metric Selector[Metric]** to a slicer.
2. Add a bar chart, line chart, or matrix.
3. Add **Selected Metric Value** to the visual values.
4. Use the slicer to switch metrics.
5. Compare this behavior to the native field parameter visual.

Reflection questions

Question	Notes
Which pattern was easier to create?	Field parameter or disconnected selector?
Which pattern was easier to understand?	Why?
Which pattern gives more DAX control?	Think about custom fallback logic.
Which pattern works better for switching dimensions?	Field parameters usually win here.

Question	Notes
What formatting issue appears when switching between currency, whole numbers, and percentages?	A single measure has one format unless you add more advanced logic.

Expected learning

Students should understand that Power Query can create the disconnected selector table, DAX is still needed to respond to slicer selections, field parameters are better for native field/measure swapping, and disconnected tables plus SWITCH are useful when you want custom logic or teaching clarity.

Deeper Understanding Challenge: Build a disconnected margin target selector

This challenge uses a disconnected what-if table to let report users choose the margin target that defines good, near-target, and below-target performance.

Goal

Create a margin target slicer using Power Query and DAX, then use it to drive KPI status and exception analysis.

Step 1: Create the disconnected what-if table in Power Query

1. Open **Transform data**.
2. Create a **Blank Query**.
3. Rename it **Margin Target**.
4. Open **Advanced Editor**.
5. Replace the query with:

```
let
    Source =
        #table(
            type table [MarginTargetLabel = text, MarginTargetValue = number, SortOrder = Int64.Type],
            {
                {"20%", 0.20, 0},
                {"25%", 0.25, 1},
                {"30%", 0.30, 2},
                {"35%", 0.35, 3},
                {"40%", 0.40, 4}
            }
        )
in
    Source
```

6. Select **Close & Apply**.
7. In Model view, confirm **Margin Target** has no relationships.
8. Sort **Margin Target[MarginTargetLabel]** by **Margin Target[SortOrder]**.

Step 2: Create the DAX measures

```
Selected Margin Target =  
SELECTEDVALUE ( 'Margin Target'[MarginTargetValue], 0.30 )
```

```
Gross Margin vs Target =  
[Gross Margin %] - [Selected Margin Target]
```

```
Margin Target Status =  
VAR Difference = [Gross Margin vs Target]  
RETURN  
    SWITCH (  
        TRUE (),  
        ISBLANK ( [Gross Margin %] ), "No margin data",  
        Difference >= 0.05, "Above target",  
        Difference >= 0, "At target",  
        Difference >= -0.05, "Near target",  
        "Below target"  
    )
```

```
Margin Target Status Color =  
VAR StatusValue = [Margin Target Status]  
RETURN  
    SWITCH (  
        StatusValue,  
        "Above target", "#107C10",  
        "At target", "#107C10",  
        "Near target", "#FFB900",  
        "Below target", "#D13438",  
        "#605E5C"  
    )
```

Step 3: Use it in a visual

- 1. Add `Margin Target[MarginTargetLabel]` to a slicer.
- 2. Add cards for `[Selected Margin Target]`, `[Gross Margin %]`, and `[Gross Margin vs Target]`.
- 3. Add a table or matrix with Customer, Product Category, `[Gross Margin %]`, `[Gross Margin vs Target]`, and `[Margin Target Status]`.
- 4. Use `[Margin Target Status Color]` for conditional formatting where appropriate.

Reflection questions

Question	Why it matters
Why does the target table have no relationship?	Shows disconnected table behavior.
Why does the measure still respond to the slicer?	<code>SELECTEDVALUE</code> reads the disconnected slicer context.
What happens when no target is selected?	Teaches default values.

Question	Why it matters
What happens when multiple targets are selected?	Teaches <code>SELECTEDVALUE</code> fallback behavior.
How does this improve UX?	Users define their own threshold without editing the report.

Exercise 6: Conditional formatting

Objective: Use visual formatting to highlight business meaning.

Tasks

1. Define thresholds for sales variance or gross margin percentage.
2. Apply conditional formatting to a table, matrix, or KPI visual.
3. Use colors and icons sparingly.
4. Add a note explaining threshold meaning.

Expected result

Formatting highlights exceptions without overwhelming the report.

Exercise 7: Mobile layout

Objective: Create a mobile-optimized report view.

Tasks

1. Open Mobile layout view.
2. Add the highest-value visuals.
3. Resize visuals for readability.
4. Avoid dense tables unless necessary.
5. Validate mobile layout flow.

Expected result

The report has a readable mobile experience with prioritized content.

Exercise 8: Accessibility review

Objective: Validate the report for accessibility and usability.

Tasks

1. Add alt text for important visuals.
2. Review tab order.
3. Check color contrast.
4. Confirm visual titles and labels are descriptive.
5. Verify that color is not the only way meaning is conveyed.

Expected result

The report includes documented accessibility improvements.

Optional: Personalized visuals

Azure Government note: Personalized visuals are marked **Verify for Gov.** Confirm Service availability and tenant settings before making this hands-on.

Discuss or demonstrate personalized visuals only after validation.

Optional: AI visuals

Azure Government note: AI visuals are marked **Verify for Gov / Commercial-focused**. Provide a non-AI alternate path for Azure Government delivery.

Discuss or demonstrate AI visuals only after validation.

Validation checklist

- ☐ Report pages have clear audience and purpose.
- ☐ Drillthrough works from summary to detail.
- ☐ Tooltip page is configured and assigned.
- ☐ Bookmarks capture only intended behavior.
- ☐ Navigation buttons are consistent.
- ☐ Field parameter switches the intended measures or dimensions.
- ☐ Conditional formatting has documented thresholds.
- ☐ Mobile layout is readable.
- ☐ Accessibility review is complete.
- ☐ Personalized visuals and AI visuals have Gov notes.